HTF: A Token-Free Language Model with Morphological CNN for Brazilian Portuguese

João L. Ortiz*, Ivan R. Guilherme*, Jonas Queiroz†
*Universidade Estadual Paulista (UNESP), São Paulo, Brazil
joao.ortiz@unesp.br, ivan.guilherme@unesp.br
†Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil
jonasq@unicamp.br

***Abstract*—Language models based on subword tokenization (BPE, WordPiece) exhibit well-known limitations, including fixed vocabularies, sensitivity to textual noise, and excessive fragmentation in morphologically rich languages. In this work, main contribution is the HTF (Hierarchical Token-Free) architecture, which imposes explicit hierarchy on sub-lexical processing via a multi-scale morphological CNN with intra-word attention, eliminating the dependence on fixed vocabularies. It is a 144M-parameter architecture that operates directly on characters, employing a multi-scale CNN with intra-word attention as a morphological feature extractor. The model is evaluated on Brazilian Portuguese, demonstrating superior robustness to textual perturbations (typos) when compared to BPE-based models of similar scale. In controlled experiments with 10% corruption rates, HTF exhibits a cross-entropy degradation of only +105%, compared to +692% for Pythia-160M and +1980% for GPT-2 PT, representing a 7× to 19× improvement in robustness. In the Cloze test, the model retains 93.8% of the correct token probability under 10% noise, compared to 71.1% for Pythia and 43.7% for GPT-2. On downstream tasks with noisy text, the model exhibits an accuracy degradation of only 1.4% under 30% corruption, compared to 6.8% for a BPE model of comparable scale.**

***Index Terms*—natural language processing, token-free language model, character-level modeling, textual noise robustness, Brazilian Portuguese**

I. Introduction

The vast majority of current language models rely on subword tokenization algorithms, such as BPE (Byte Pair Encoding) [1], WordPiece [2], or Unigram [3], to convert text into integer sequences prior to neural processing. Although computationally efficient, these methods introduce fundamental limitations, specially in morphological rich languages. In the present work, we will use Brazilian Portuguese as the main language:

1) **Fixed vocabulary**: tokens are defined during pretraining and do not adapt to new domains. Technical terms, neologisms, and proper nouns are fragmented into sub-tokens devoid of individual meaning.
2) **Noise fragility**: a single altered character can produce entirely different tokens. The word "energia" ("energy" in Portuguese), tokenized as a single token, becomes "ener" + "gia" after a typo, destroying the semantic representation.
3) **Morphological inefficiency**: in morphologically rich languages such as Portuguese, where a single verb can have dozens of inflected forms, BPE frequently fragments words inconsistently across inflections sharing the same stem.

Recent work on byte-level models, such as BLT [4], SpaceByte [5], and MambaByte [6], demonstrates that is feasible to operate directly on bytes without significant performance loss

However, these models treat bytes as structureless atomic units, disregarding the natural hierarchy of written language: characters form morphemes, which form words, which form sentences.

In this work, we explore a distinct approach: rather than observing and iterating over individual bytes, we operate on *words as sequences of characters*, employing a multi-scale CNN (Convolutional Neural Network) with intra-word attention to extract explicit morphological representations. We refer to this approach as *token-free* in the sense that it does not employ learned subword tokenization: there is no statistical vocabulary (BPE, WordPiece), nor corpus-dependent segmentation. The mapping from characters to integer indices is a fixed and exhaustive encoding (analogous to the UTF-8 table in byte-level models), and word segmentation is deterministic via regular expression. The central hypothesis is that this hierarchical decomposition, where the CNN captures intra-word structure and the Transformer captures inter-word relations, yields representations that are more robust to character-level perturbations. In this work, we explore a distinct approach: rather than observing and iterating over individual bytes, we operate on *words as sequences of characters*, employing a multi-scale CNN with intra-word attention to extract explicit morphological representations. We refer to this approach as *token-free* in the sense that it does not employ learned subword tokenization: there is no statistical vocabulary (BPE, WordPiece), nor corpus-dependent segmentation. The mapping from characters to integer indices is a fixed and exhaustive encoding (analogous to the UTF-8 table in byte-level models), and word segmentation is deterministic via regular expression. The central hypothesis is that this hierarchical decomposition, where the CNN captures intra-word structure and the Transformer captures inter-word relations, yields representations that are more robust to character-level perturbations. In this sense, the main contribution of this work is the HTF architecture, which imposes explicit hierarchy on sub-lexical processing via a multi-scale morphological CNN with intra-word attention,

thereby eliminating the dependence of language models on fixed vocabularies.

This paper is organized as follows: Section II reviews related work; Sections III and IV present the architecture and the proposed approach; Section V discusses the experimental results; and Sections VII and VIII present the conclusions.

II. Related Work

A. Byte-Level Models

BLT (Byte Latent Transformer) [4] proposes dynamic entropy-based patches, grouping bytes into variable-length units. SpaceByte [5] applies larger Transformer blocks only at word boundaries, concentrating computational capacity on predicting the first character of each word. MambaByte [6] employs State Space Models to process long byte sequences with fixed memory. ByT5 [7] demonstrates that the T5 architecture can operate directly on UTF-8 bytes with competitive performance.

All of these works operate at the level of individual bytes (UTF-8 codes, 256 possible values) and rely on the model to implicitly learn morphological structure. Our approach differs in two aspects: (1) it operates at the level of complete Unicode *characters*, not bytes, so that an accented character such as "ã" constitutes a single unit rather than two separate bytes; and (2) it makes morphological structure *explicit* through a CNN with multiple filters of different fixed sizes, each capturing character n-grams at a particular scale.

B. Character-Level Models with CNN

The use of CNNs to process characters has precedents in works such as CharCNN [8] and ELMo [9], which employ convolutions over characters to produce word representations. In particular, ELMo applies 2048 character n-gram convolutional filters followed by highway layers and linear projection to generate one embedding per token, which is then contextualized by biLSTMs. Our work differs in three aspects: (1) we add *cross-scale self-attention* within the CNN, allowing filters of different sizes to share information before aggregation; (2) we use max-pooling as a noise filtering mechanism instead of highway layers; and (3) HTF is a complete generative language model (encoder + decoder + character-by-character generation), not merely an embedding extractor for downstream tasks.

C. Robustness to Textual Noise

The fragility of BPE models to textual perturbations is documented in [10], [11]. Character-level models demonstrate greater natural robustness [7], [12], but typically at higher computational cost. Our work quantifies this robustness in a controlled manner, comparing directly against BPE baselines under identical experimental conditions.

III. Hierarchical Token-Free

HTF (Hierarchical Token-Free) comprises four hierarchical modules that process text from the finest granularity (characters) to the coarsest (word sequences), and then generate text in the reverse direction. The designation *hierarchical* refers to the explicit decomposition into granularity levels: (1) the *WordEncoder* operates at the intra-word level, extracting morphological representations from character n-grams via multi-scale CNN; (2) the *Encoder Transformer* operates at the inter-word level, contextualizing word representations within the sequence; (3) the *Decoder Transformer* generates latent representations for each future position; and (4) the *WordDecoder* materializes each latent representation into a character sequence. The flow is therefore characters → word → sequence (encoding) and sequence → word → characters (decoding), with each level operating at a distinct scale of language. Figure 1 illustrates this organization.

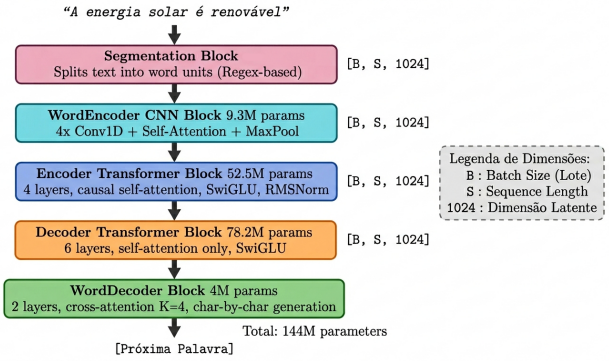


Figure 1: HTF architecture. Text is segmented into words via regex and each word is independently processed by the WordEncoder (multi-scale CNN + intra-word attention + max-pooling), producing a 1024-dimensional vector per word. The sequence of word-vectors is contextualized by the Encoder Transformer (4 layers) and then processed by the Decoder Transformer (6 layers, self-attention only), which produces latent representations for each position. The WordDecoder (2 layers, cross-attention to context) materializes each representation into a character sequence via local autoregressive generation.

A. Segmentation Block

The input text is segmented into units by a regular expression (`\S+\s*`), where each unit corresponds to a word followed by its trailing whitespace. This segmentation is deterministic and vocabulary-independent. The segmentation mechanism itself is language-agnostic; however, the current character vocabulary (155 symbols) specifically covers the Latin alphabet with Portuguese diacritics, and is extensible to other languages by expanding the character table without architectural modifications.

B. WordEncoder: Multi-Scale CNN with Attention

Each unit (word) is independently processed by the CNN (9.3M parameters). Figure 2 illustrates an example of the processing pipeline:

1) **Character embedding**: each character is mapped to a 64-dimensional vector via an embedding table ($|V| =$

155 characters, covering uppercase and lowercase Latin alphabet, digits, Portuguese diacritics – acute accents, circumflexes, graves, tilde, cedilla, e.g: punctuation, and special tokens). Characters outside this vocabulary are mapped to the <UNK> symbol. Figure 2 illustrates how the word "energia" is processed at this stage, yielding a matrix of dimension 7×64.

2) **Multi-scale convolutions**: four banks of 1024 filters with kernel sizes 2, 3, 4, and 5 are applied in parallel, capturing character bigrams, trigrams, tetragrams, and pentagrams. In total, 4096 individual 1D filters are applied to each word. For a word of L characters, each kernel bank of size k produces $L - k + 1$ feature maps of 1024 dimensions, yielding $4L - 10$ feature maps per word. For simplicity, we henceforth refer to each feature map as a token, since from this point we obtain our basic word units, e.g: not fixed values as in BPE, but dense representations of all sub-parts of the word. Figure 2-left illustrates the four filters and the resulting token feature maps for the given example.
3) **Non-linear projection**: each convolutional feature map undergoes a linear transformation followed by $\tanh$, projecting the representations from all four scales into a common 1024-dimensional space. This operation is applied independently to each scale, prior to concatenation.
4) **Concatenation**: the feature maps from all four scales are concatenated into a single sequence. For a word of L characters, the result is a matrix $[4L - 10, 1024]$ containing all detected n-grams across all scales.
5) **Intra-word attention**: the concatenated sequence (between 18 and 30 positions per word in Portuguese) is processed by 16 self-attention heads, enabling information from different scales to interact (e.g., the prefix detected by kernel 3 attends to the suffix detected by kernel 5). The output maintains the same dimensionality $[4L - 10, 1024]$, but with cross-scale information exchange.
6) **Aggregation**: a max-pooling layer (Figure 2) is applied over all positions to produce a final 1024-dimensional vector per word. It acts as a noise filter, retaining only the n-gram activations that were relevant within the word.

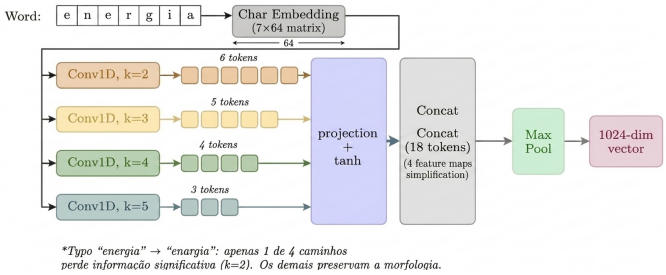


Figure 2: Processing of the word "energia" (7 characters) by the WordEncoder. From left to right: character embedding [7, 64], multi-scale convolutions with kernels 2–5 applied in parallel, projection + tanh, concatenation [18, 1024], cross-scale self-attention (16 heads), and max-pooling to the final vector [1024].

C. Encoder Transformer

While the WordEncoder captures intra-word morphological structure (character n-grams), the Encoder Transformer operates at the inter-word level: its role is to contextualize word representations within the sequence, capturing syntactic and semantic relations between positions. The sequence of word-vectors is processed by a 4-layer causal Transformer (52.5M parameters):

- Learned positional embedding (up to 1024 positions)
- Causal self-attention with 16 heads
- SwiGLU feedforward [13] ($d_{ff} = 2816$)
- RMSNorm [14] (pre-norm) and residual dropout (0.1)
- Per-layer gradient checkpointing

D. Decoder Transformer

A 6-layer causal Transformer (78.2M parameters) with self-attention only was chosen, without cross-attention to the encoder. The removal of cross-attention was an empirical decision: in language modeling, where input and output are the same sequence, cross-attention created a copy shortcut that caused mode collapse. It includes a linear head for end-of-sequence (EOS) prediction. Figure 3 illustrates the complete flow of the Decoder Transformer and WordDecoder, highlighting the two decision hierarchies: EOS prediction (sequence level) and autoregressive character generation (word level).

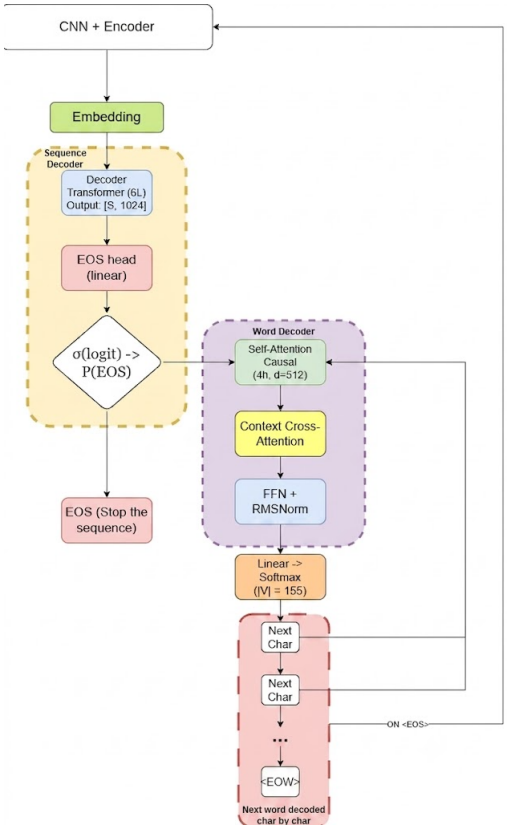


Figure 3: Decoder flow: the Decoder Transformer (6L) produces 1024-dimensional representations per position. At each position, the EOS head decides whether the sequence terminates. Otherwise, the WordDecoder generates the word character-by-character via local autoregression, returning to the sequence level only after producing <EOW>. The CNN + Encoder block referenced at the top corresponds to Figure 2.

E. WordDecoder: Local Transformer

As illustrated in Figure 3-right, it Generates characters word-by-word via a 2-layer Transformer (4M parameters) with:

- Causal self-attention (4 heads, $d = 512$)
- Cross-attention to a window of $K = 4$ positions from the Decoder Transformer
- Global context projection ($1024 \to 512$)
- Output: distribution over the character vocabulary

Teacher forcing is employed during training.

F. Algorithm

Algorithm 1 describes the HTF inference procedure, illustrating the autoregressive generation loop with the two-level hierarchy (sequence-level EOS decision and word-level character generation). Algorithm 2 details the forward pass during training, where teacher forcing replaces the autoregressive loops.

Require: Input text (prompt) T
Ensure: Generated text G

$W \leftarrow \text{regex}(T, \texttt{\textbackslash S+\textbackslash s*})$ {Segment prompt into words}
$G \leftarrow []$

// Encode prompt
for each word W_i of L characters **do**
  $\mathbf{w}_i \leftarrow \text{WordEncoder}(W_i)$ {CNN + Attention + MaxPool $\to$ [1024]}
end for
$H \leftarrow \text{EncoderTransformer}([\mathbf{w}_1, ..., \mathbf{w}_S])$ {4L causal}

// Autoregressive generation
while not EOS **do**
  $D \leftarrow \text{DecoderTransformer}(H)$ {6L, self-attention only}
  $p_{\text{eos}} \leftarrow \sigma(\text{Linear}(D_{-1}))$ {EOS probability at last position}
  if $p_{\text{eos}} > 0.5$ **then**
    break {End of sequence}
  end if

  // WordDecoder: generate word character-by-character

  $\text{ctx} \leftarrow \text{Linear}_{1024\to512}(D_{-K:})$ {Window $K=4$}
  $c_0 \leftarrow$ <SOS>
  while $c_t \neq$ <EOW> and $t < 30$ **do**
    $X \leftarrow \text{CharEmbed}([c_0, ..., c_t]) + \text{PosEmbed}$
    $X \leftarrow \text{WordDecoderLayers}(X, \text{ctx})$ {2L, self+cross attention}
    $c_{t+1} \leftarrow \arg\max(\text{Linear}(X_{-1}))$ {Sample from [155]}
  end while
  word $\leftarrow [c_1, ..., c_t]$
  Append word to G
  $H \leftarrow H \oplus \text{WordEncoder}(\text{word})$ {Extend context}
end while
return G

Algorithm 1: HTF inference (autoregressive generation).

Require: Input text T
Ensure: Character logits, EOS logit

Segmentation: $W \leftarrow \text{regex}(T, \texttt{\textbackslash S+\textbackslash s*})$ {Words}
Char encoding: $C_i \leftarrow \text{char_to_idx}(W_i)$ for each word i

// WordEncoder (per word, in parallel)
for each word C_i of L characters **do**
  $E \leftarrow \text{CharEmbed}(C_i)$ $\{[L, 64]\}$
  for $k \in \{2, 3, 4, 5\}$ **do**
    $F_k \leftarrow \tanh(\text{Linear}(\text{ReLU}(\text{Conv1D}_k(E))))$ $\{[L-k+1, 1024]\}$
  end for
  $F \leftarrow \text{Concat}(F_2, F_3, F_4, F_5)$ $\{[4L-10, 1024]\}$
  $F \leftarrow F + \text{SelfAttention}_{16h}(\text{RMSNorm}(F))$
  $\mathbf{w}_i \leftarrow \text{MaxPool}(F)$ $\{[1024]\}$
end for

// Encoder Transformer
$H \leftarrow [\mathbf{w}_1, ..., \mathbf{w}_S] + \text{PosEmbed}$ $\{[S, 1024]\}$
for $l = 1$ to 4 **do**
  $H \leftarrow \text{TransformerLayer}(H, \text{causal_mask})$
end for

// Decoder Transformer
$D \leftarrow H + \text{PosEmbed}$
for $l = 1$ to 6 **do**
  $D \leftarrow \text{TransformerLayer}(D, \text{causal_mask})$
end for
eos_logit $\leftarrow \text{Linear}(D)$ {P(EOS) per position}

// WordDecoder (per position, teacher forcing)
ctx $\leftarrow \text{Linear}_{1024 \to 512}(D)$ {Window $K=4$}
for each position i **do**
  input $\leftarrow [\texttt{<SOS>}, c_1, ..., c_{N-1}]$ {shifted targets}
  $X \leftarrow \text{CharEmbed}(\text{input}) + \text{PosEmbed}$
  for $l = 1$ to 2 **do**
    $X \leftarrow \text{SelfAttn}(X) + \text{CrossAttn}(X, \text{ctx}_i)$
    $X \leftarrow X + \text{FFN}(X)$
  end for
  $\text{logits}_i \leftarrow \text{Linear}(X)$ $\{[N, 155]\}$
end for
return logits, eos_logit

Algorithm 2: HTF forward pass during training.

IV. Training

A. Data

The model is first trained on a mixture of three datasets: Carolina [15], the Portuguese subset of Wikimedia Wikipedia [16], and the Portuguese portion of FineWeb-2 [17]. We then perform a refinement stage using the GigaVerbo dataset [18], a 200-billion-token Portuguese corpus (in the BPE sense) compiled from several sources (curated web, Wikipedia, news articles, translated dialogues, and legal documents). From GigaVerbo, we employ only those samples labeled as high quality (label $=$ 1) by a BERT-based filter specifically trained to assess Portuguese text quality, yielding roughly 95M training instances.

B. Pre-training

Pre-training was conducted in two phases with distinct data and hyperparameter configurations:

Phase 1 (steps 0 to 99k, Wikipedia+Carolina+FineWeb-2 data):

The objective of this phase is to establish the CNN morphological representations and basic language modeling capability. We employ diversified data (encyclopedic, academic, and web) to expose the model to a broad variety of Portuguese linguistic structures. The differentiated learning rate between encoder and decoder reflects the hypothesis that the WordEncoder (CNN) requires more conservative updates due to operating on finer-grained representations. Additionally, decoupling the learning rates avoids cyclic instability: if both modules update at the same rate, the decoder adapts to encoder representations that are simultaneously changing, creating a pursuit cycle that hinders convergence.

- **Optimizer**: AdamW [19] ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay $= 0.01$)
- **Learning rate**: Encoder 1×10^{-4}, Decoder 1.6×10^{-4}
- **Scheduler**: Linear warmup (1000 steps) + cosine decay to step 100k
- **Dropout**: attention only (0.1)

Phase 2 (steps 99k to 134k, GigaVerbo data):

After Phase 1 convergence, the objective of this phase is to refine generation quality using a higher-quality corpus (GigaVerbo, filtered by BERT classifier). The learning rate is reduced to prevent degradation of already-learned representations, and dropout is expanded to all layers to prevent overfitting to the more homogeneous corpus. The scheduler with warm restarts allows the model to escape local minima during adaptation to the new data domain.

- **Reduced learning rate**: Encoder 8×10^{-5}, Decoder 1.2×10^{-4}
- **Restarted scheduler**: new warmup + cosine with warm restarts (50k-step cycles, peak decays 70% per cycle, minimum 10% of peak)
- **Expanded dropout** (0.1): added to feedforward layers (SwiGLU) and residual connections
- **Adaptive safeguard**: if validation loss rises more than 8% above the best, the LR is automatically decayed to prevent instability

Both phases employ:

- **Effective batch size**: 32 (batch 2 $\times$ accumulation 16)
- **Mixed precision**: AMP (float16) with GradScaler
- **Gradient clipping**: maximum norm 0.5
- **Auxiliary contrastive loss** (weight 0.05): encourages local coherence in encoder embeddings by making adjacent positions more similar than positions from different texts (margin 0.2). This auxiliary loss is specific to the HTF architecture and aims to stabilize WordEncoder representations during joint training with the decoder

- **Teacher forcing** in the WordDecoder throughout pre-training: at each step, the WordDecoder receives the correct characters as input (rather than its own predictions), accelerating convergence of character-by-character generation – an established approach for autoregressive decoders

The remaining optimization choices follow the standard established by the Llama model family [20], adopted as they represent the state of the art in autoregressive language model training.

C. Convergence History

Model development comprised an exploratory phase and two effective pre-training phases. The exploratory phase employed a simplified CNN (0.2M parameters, without intra-word attention) as a proof of concept, which was discarded after 25.5k steps. Effective pre-training begins with the final architecture (CNN with attention, 9.3M parameters) with reinitialized weights at step 0 – therefore, the model reported in this work was trained for 109k effective steps (30.5k to 134.5k in Table I, excluding the PoC (Proof Of Concept)).

Table I records the evolution of the validation loss throughout development. Row 1 (initial CNN) and row 2 (CNN with attention) correspond to the exploratory phase: both used the same data (Wikipedia+Carolina+FineWeb-2) and Phase 1 hyperparameters, but the transition between them involved weight reinitialization due to the CNN redesign – hence, the data seen in row 1 was re-presented in row 2. From row 2 onward, training proceeded without reinitialization, always with new data from the same corpus until step 98.5k (row 3, end of Phase 1). Row 4 corresponds to Phase 2, with the switch to GigaVerbo and the regularization adjustments described above.

Table I: Validation loss evolution during the model development

Phase	Step	Val Loss	Change
Initial CNN (0.2M, PoC)	25,500	2.734	Baseline
CNN with attention (9.3M)	30,500	2.515	CNN redesigned (weights reinitialized)
+ Scheduler/Dropout	98,500	0.816	LR restart + dropout
+ GigaVerbo	134,500	0.579	High-quality data

The largest absolute drop resulted from the combination of the LR scheduler with warm restarts and expanded dropout, which reduced the validation loss from 2.515 to 0.816 over 70k steps. The switch to GigaVerbo produced the fastest improvement, reducing the validation loss from 0.816 to 0.579 in a few thousand steps, evidencing the impact of data quality.

V. Experiments

A. Typo Robustness Benchmark

To evaluate robustness to textual perturbations, we employ a controlled protocol:

Data: 200 sentences from the ASSIN2 dataset [21] (test split), with a minimum length of 30 characters.

Corruption: Synthetic typos applied at rates of 0%, 5%, 10%, 20%, and 30% of alphabetic characters. Four operation types with uniform probability: adjacent-key substitution (QWERTY layout), deletion, adjacent-character insertion, and transposition. Each sentence is corrupted with 3 different seeds.

Metrics:

- **ΔCE**: relative variation of cross-entropy loss between clean and corrupted text. Lower values indicate greater robustness.
- **Generation consistency**: given a prompt (first half of the sentence), the overlap of the first 3 generated tokens between clean and corrupted prompts is measured.
- **Cloze**: retention of the correct token probability when the context is corrupted.

Baselines: GPT-2 PT (124M, BPE) [22] and Pythia-160M (BPE) [23], evaluated under identical conditions.

B. Consolidated Results and Analysis

Tables II, III, IV and Figure 4 present the results of the experiments.

Table II: Typo robustness, relative ΔCE (%) by corruption rate

Model	0%	5%	10%	20%	30%
GPT-2 PT	–	+676%	+1980%	+2732%	+2126%
Pythia-160M	–	+211%	+692%	+1632%	+2115%
HTF	–	**+50%**	**+105%**	**+177%**	**+214%**

Table III: Generation consistency (%) by corruption rate

Model	0%	5%	10%	20%	30%
GPT-2 PT	100%	39.6%	27.6%	17.2%	10.4%
Pythia-160M	99.3%	45.2%	40.0%	22.6%	16.2%
HTF	91.6%	40.2%	28.8%	17.5%	10.7%

Table IV: Cloze, correct token probability retention (%)

Model	0%	5%	10%	20%	30%
GPT-2 PT	100%	72.9%	43.7%	17.7%	6.3%
Pythia-160M	100%	91.2%	71.1%	47.3%	31.1%
HTF	100%	**98.3%**	**93.8%**	**88.5%**	**81.0%**

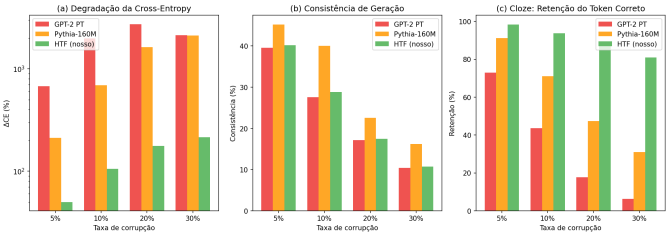


Figure 4: Robustness comparison: ΔCE by corruption rate

The superiority of HTF in perplexity robustness and Cloze is explained by the architecture:

1) **N-gram redundancy**: when a typo alters a character in the word "energia" to "enargia", the convolutional filters of size 3 still detect "ene", "nar", "rgi", "gia", i.e., 3 out of 4 trigrams survive intact. In contrast, the BPE tokenizer may produce entirely different tokens for the corrupted word.
2) **Max-pooling as a noise filter**: aggregation via max-pooling selects the strongest activations, naturally disregarding filters that did not find recognizable patterns in the corrupted text.
3) **Intra-word attention as correction**: cross-scale self-attention allows information from intact n-grams to compensate for the loss of corrupted n-grams.

However, in the generation consistency metric (see Table III, HTF (28.8%) falls below Pythia (40.0%), although above GPT-2 PT (27.6%). This is attributable to two factors: (1) the model is still in the training phase (134k steps), with the WordDecoder not fully converged, introducing instability in generation; and (2) the CNN architecture propagates small input variations non-linearly, such that a perturbation that does not affect *comprehension* (Cloze retains 93.8%) may nonetheless alter the *choice* of the next generated word. This dissociation between robust comprehension and sensitive generation suggests that HTF's robustness resides primarily in the encoder (CNN + Transformer), while the decoder remains vulnerable to variations in input representations.

VI. Multi-task Fine-tuning and Comparison with BPE Baseline

The experiments in Section V demonstrate HTF's robustness in pure language modeling. To evaluate whether this advantage transfers to downstream tasks, we conduct a multi-task supervised fine-tuning (SFT) experiment under controlled conditions, comparing directly against Tucano-160M [18], a decoder-only model based on the Llama architecture [20] with BPE tokenization.

A. Experimental Setup

Both models are comparable in scale. Table V lists their features.

Table V: Architectural comparison of evaluated models

	HTF	**Tucano-160M**
Parameters	144M	162M (+11%)
Tokenization	Character (open)	BPE 32k (fixed)
Encoder	CNN + 4L Transformer	–
Decoder	6L Transformer	12L Llama
Generation	Char-level (2L local)	Softmax over BPE vocab
Pre-training	From scratch, GigaVerbo and others, 109k steps	GigaVerbo, 169B tokens

Fine-tuning employs **exactly the same data and hyperparameters** for both models:

- **Dataset**: SST-2 (∼64k examples), BoolQ (∼9k), RTE with varied explanations (∼2.6k), ASSIN2-RTE (∼6.2k), NER LeNER-Br and NER HAREM (∼10k) – total ∼92k examples (NER was included as a correlated task to enhance training).
- **Steps**: 30,000
- **Effective batch size**: 16 (batch 4 × gradient accumulation 4)
- **Learning rate**: 2×10^{-5} (AdamW)
- **Selection criterion**: checkpoint with lowest validation loss

The prompt format differs only in each model's native delimiters: ChatML (`<|im_start|>`) for HTF and `<instruction>` for Tucano.

B. Evaluation Tasks

Evaluation employs the **official validation splits** of each dataset, never seen during training:

- **SST-2**: binary sentiment analysis (positive/negative), 872 examples
- **BoolQ**: yes/no question about a paragraph, 3,270 examples
- **RTE**: textual entailment – is the hypothesis supported by the premise?, 277 examples
- **ASSIN2-RTE**: textual entailment native to PT-BR – does sentence 2 follow from sentence 1?, 500 examples

C. Fine-tuning Results

Table VI presents the validation loss progression during SFT, and Table VII reports per-task accuracy at the best checkpoint.

Table VI: Validation loss during the SFT training

Step	**HTF**	**Tucano-160M**
4.000	**0.985**	1,591
2.000	**0.981**	1,629
8.000	**1.000**	1,553
16,000	**1.027**	1,524
30,000	**1.065**	1,507
Best	**0.981** (step 2k)	1,507 (step 30k)

1) Convergence: HTF reaches the lowest val_loss in only 2,000 steps (∼0.7 epochs), while Tucano has not yet converged after 30,000 steps (monotonically decreasing val_loss). Even at Tucano's best point (1.507), the val_loss is 54% higher than HTF's (0.981), indicating that the hierarchical representation of HTF learns the instruction→response mapping with significantly less data exposure.

2) Per-Task Accuracy: Table VII reports the accuracy on official validation splits using the checkpoint with the lowest validation loss for each model.

D. Results Analysis

The results reveal a complementarity between the architectures:

1) **HTF superior in textual entailment**: on NLI tasks (RTE: +6pp, ASSIN2: +13pp), HTF outperforms Tucano. These tasks require comparing sentence pairs and

Table VII: Accuracy on official validation splits (lowest val_loss checkpoint)

Task	HTF	Tucano	N	Winner
SST-2	74.05%	**88.17%**	872	Tucano
BoolQ	57.41%	**60.95%**	3.270	Tucano
RTE	**56.04%**	50.00%†	277	HTF
ASSIN2	**50.00%**	37.40%	500	HTF
Average	**59.38%**	59.13%	–	Tie

† Tucano produced recognizable responses in only 24 of 277 examples (8.7%); 253 responses were discarded for not containing extractable classification indicators.

determining logical relations, benefiting from the explicit morphological representation that preserves intra-word structure.

2) **Tucano superior in lexical classification**: on SST-2 (+14pp) and BoolQ (+3.5pp), Tucano benefits from 169B tokens of BPE pre-training, which create richer lexical embeddings for tasks that depend on pattern recognition in individual words.
3) **Format generalization**: HTF responded in a recognizable format in 99% of RTE examples (273/277), while Tucano responded in only 8.7% (24/277). This indicates that HTF better generalizes the response structure learned during SFT to prompt styles not seen exactly during training.
4) **Data efficiency**: HTF converged in $\sim$2,000 steps versus $>$30,000 for Tucano, demonstrating that the hierarchical representation requires fewer examples for fine-tuning.

These results are consistent with the observation by Corrêa et al. [18], who report a *negative* correlation ($r = -0.30$) between pre-training volume and ASSIN2-RTE performance for Tucano-160M, suggesting that BPE models at this scale have intrinsic difficulty with textual entailment tasks.

E. Typo Robustness in Fine-tuned Tasks

To verify whether the noise robustness demonstrated in Section V is preserved after fine-tuning and translates into practical advantage, we repeat the evaluation of the four tasks with synthetic perturbations applied to the input data.

Protocol: The same validation splits are corrupted with typos at rates of 0%, 5%, 10%, 20%, and 30% (same mechanism as in Section V: QWERTY substitution, deletion, insertion, and transposition). Each example is evaluated with 3 corruption seeds and the final prediction is decided by majority vote. We use a subset of 200 examples per task. The results are reported in Tables VIII and IX.

1) Robustness Analysis: The results confirm that the intrinsic robustness of the CNN architecture fully transfers to downstream tasks:

1) **HTF nearly immune to typos**: from 0% to 30% corruption, HTF's mean accuracy varies by less than 1 percentage point (60.33% $\rightarrow$ 59.50%), a degradation of only 1.4%. The model maintains discriminative capacity even with nearly one-third of characters corrupted.

Table VIII: Per-task accuracy under textual perturbation (%)

Task	Model	0%	5%	10%	20%	30%
SST-2	HTF	75.50	78.00	**79.00**	**72.50**	**73.50**
	Tucano	**87.50**	**86.50**	79.00	69.50	62.50
BoolQ	HTF	62.00	61.50	**61.50**	**65.00**	**62.50**
	Tucano	**63.00**	**62.00**	59.00	59.00	59.50
RTE	HTF	**55.33**	**52.76**	**53.00**	**52.26**	**52.50**
	Tucano	47.37	43.90	37.78	50.77	51.67
ASSIN2	HTF	**48.50**	**48.50**	**48.50**	48.50	**49.50**
	Tucano	40.50	44.00	45.00	**49.50**	48.50

Table IX: Mean Accuracy (4 tasks) and relative degradation under typos

Model	0%	5%	10%	20%	30%	Δ30%
HTF	**60.33%**	**60.19%**	**60.50%**	**59.57%**	**59.50%**	**−1.4%**
Tucano-160M	59.59%	59.10%	55.19%	57.19%	55.54%	−6.8%

2) **Tucano degrades 5$\times$ more**: Tucano loses 6.8% of mean accuracy at 30% corruption, with particularly severe degradation on SST-2 ($-$25pp, from 87.5% to 62.5%). BPE tokenization, sensitive to individual character alterations, propagates errors destructively.
3) **Advantage inversion**: on clean text, Tucano outperforms HTF on SST-2 by 12pp (87.5% vs 75.5%). However, at 10% corruption both tie (79%), and at 30% HTF surpasses by 11pp (73.5% vs 62.5%). At 30% corruption, HTF outperforms Tucano on *all* four tasks.
4) **Stability on NLI**: HTF maintains stable performance on RTE across all rates (55%$\rightarrow$52.5%, $-$2.8pp), while Tucano exhibits high variance (47%$\rightarrow$38%$\rightarrow$51%), indicating that its predictions become effectively random under noise – converging to chance ($\sim$50%) on a binary task.

These results have direct practical implications: for applications that process noisy text (manual reports, social media analysis, OCR, ASR transcriptions), HTF offers more reliable and predictable performance than BPE models of equivalent scale. The robustness demonstrated in language modeling (Section V) is not only preserved but translates into measurable functional advantage on downstream tasks.

VII. Discussion

A. Advantages for Morphologically Rich Languages

Portuguese exhibits rich inflectional morphology: a verb such as "correr" (to run) generates dozens of forms (corro, corres, corremos, correria, corrêramos, among others). In BPE models, each form may be tokenized differently and inconsistently. In HTF, all forms share the same stem n-grams ("corr", "orre"), producing naturally similar representations without the need for explicit learning of this relationship.

B. Limitations and Perspectives

The identified limitations do not compromise the model's practical utility, but delimit its current scope:

- **Generation cost**: the main computational bottleneck of HTF resides in the two-level autoregressive generation: the Decoder Transformer generates representations word-by-word, and the WordDecoder materializes each word character-by-character. With KV-cache implemented in the Decoder, HTF achieves 36.3 words/s (198 chars/s), being 2.4× slower than Tucano-160M (87.2 words/s) under identical hardware conditions. Without KV-cache, this gap increases to 7.3× (11.9 vs 87.2 words/s). The WordEncoder (CNN) overhead is negligible in comparison: all prompt words are processed in parallel in a single pass, and the actual bottleneck is the dual autoregressivity during generation. Additional optimizations such as character-level *speculative decoding* (where multiple candidate characters are generated in parallel by a draft model and verified in batch by the main model, accepting all those confirmed in a single forward pass) or parallel multi-character prediction (similar to Multi-Token Prediction [24]) are viable directions to reduce this gap without architectural modifications.
- **Max-pooling as aggregation**: compressing n-gram vectors into a single vector per word discards intra-word positional information. However, for reference, BERT compresses entire sentences of up to 512 tokens into 768-dimensional vectors. Representing a single word (typically 5 to 8 characters, generating 18 intermediate vectors) in 1024 dimensions provides abundant representational capacity. Inter-word contextual information is captured by the Encoder Transformer in subsequent layers. Alternatives such as attention-pooling could be explored, but in the present work this compression did not constitute a bottleneck in the current configuration.
- **Word-level context**: with MAX_SEQ_LEN = 1024 words, the effective context is approximately 5,000 to 6,000 characters. In Portuguese, the typical ratio is 1.3 to 1.5 BPE tokens per word, so a BPE model with a 1024-token context covers approximately $1024 \div 1.4 \approx 730$ words. HTF, with 1024 words of context, therefore, offers approximately 40% more textual coverage than a BPE model with the same sequence length. This is a configuration limitation, not an architectural one: the model can be trained with longer sequences without architectural modifications.
- **Factual knowledge**: with 144M parameters, the model prioritizes fluency and linguistic structure over factual memorization. This is an expected scale limitation, compounded by the absence of a look-up table in the architecture. HTF is particularly suited for applications where linguistic robustness matters more than factual precision: text classification, sentiment analysis, intent detection, noisy text processing (OCR, transcriptions, social media), and in RAG (Retrieval-Augmented Generation) systems, where factual knowledge is provided by retrieved context – as demonstrated experimentally in this work: text classification (SST-2: 74%), sentiment analysis, and noisy text processing (degradation $<2\%$ in accuracy under 30% textual corruption).

C. Future Work

- Evaluation on the BLUEX benchmark (Brazilian university entrance exams) to measure semantic comprehension
- Direct comparison with EvaByte and Llama3-2-3B-IT-Byte in robustness
- Exploration of latent (learned) segmentation instead of regex
- Scaling to 500M+ parameters with additional data

VIII. Conclusion

We present HTF, a token-free language architecture that employs a multi-scale CNN with intra-word attention to extract morphological representations directly from characters. Experimental results demonstrate that this approach is 7 to 19 times more robust to textual perturbations than BPE models of comparable size, validating the hypothesis that explicit morphological extraction confers natural resilience to noise.

For morphologically rich languages such as Portuguese, where subword tokenization is inherently suboptimal, token-free architectures with explicit morphological hierarchy represent a promising direction that warrants continued investigation.

Acknowledgments

We give our thanks Fundação para o Desenvolvimento da Unesp (Fundunesp) for the financial support of this project. We give our thanks UNESPetro for providing the computational resources necessary for the development of this project.

References

[1] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725.

[2] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey *et al.*, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.

[3] T. Kudo and J. Richardson, "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2018, pp. 66–71.

[4] A. Pagnoni, S. Ram, R. Pasunuru, M. Bansal, L. Zettlemoyer, and V. Stoyanov, "Byte latent transformer: Patches scale better than tokens," *arXiv preprint arXiv:2412.09871*, 2024.

[5] K. Slagle, "Spacebyte: Towards deleting tokenization from large language modeling," *arXiv preprint arXiv:2404.14408*, 2024.

[6] J. Wang, T. Gangavarapu, J. N. Yan, and A. M. Rush, "Mambabyte: Token-free selective state space model," *arXiv preprint arXiv:2401.13660*, 2024.

[7] L. Xue, A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel, "Byt5: Towards a token-free future with pre-trained byte-to-byte models," *Transactions of the Association for Computational Linguistics (TACL)*, vol. 10, pp. 291–306, 2022.

[8] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, p. 649–657.

[9] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018, pp. 2227–2237.

[10] V. Karpukhin, O. Levy, J. Eisenstein, and M. Ghazvininejad, "Training on synthetic noise improves robustness to natural noise in machine translation," in *Proceedings of the 5th Workshop on Noisy User-generated Text (W-NUT)*, 2019, pp. 42–47.

[11] D. Pruthi, B. Dhingra, and Z. C. Lipton, "Combating adversarial misspellings with robust word recognition," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 5582–5591.

[12] J. H. Clark, D. Garrette, I. Turc, and J. Wieting, "Canine: Pre-training an efficient tokenization-free encoder for language representation," *Transactions of the Association for Computational Linguistics (TACL)*, vol. 10, pp. 73–91, 2022.

[13] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[14] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[15] M. Finger, M. C. Paixão de Sousa, C. Namiuti, and M. L. Martins, "Carolina: a general corpus of contemporary brazilian portuguese with provenance, typology and versioning information," *arXiv preprint arXiv:2303.16098*, 2023.

[16] Wikimedia Foundation, "Wikimedia wikipedia dataset (20231101.pt)," https://huggingface.co/datasets/wikimedia/wikipedia, 2023.

[17] G. Penedo, H. Kydlíček *et al.*, "One pipeline to scale them all — adapting pre-training data processing to every language," *arXiv preprint arXiv:2506.20920*, 2025.

[18] N. K. Corrêa, A. Sen, S. Falk, and S. Fatimah, "Tucano: Advancing Neural Text Generation for Portuguese," 2024. [Online]. Available: https://arxiv.org/abs/2411.07854

[19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019.

[20] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

[21] L. Real, E. Fonseca, and H. Gonçalo Oliveira, "The assin 2 shared task: A quick overview," in *Computational Processing of the Portuguese Language: 14th International Conference, PROPOR 2020, Evora, Portugal, March 2–4, 2020, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 406–412. [Online]. Available: https://doi.org/10.1007/978-3-030-41505-1_39

[22] P. Guillou, "Gpt-2 small portuguese," https://huggingface.co/pierreguillou/gpt2-small-portuguese, 2021.

[23] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff *et al.*, "Pythia: A suite for analyzing large language models across training and scaling," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[24] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, "Better faster large language models via multi-token prediction," 2024. [Online]. Available: https://arxiv.org/abs/2404.19737